

LAPORAN PRAKTIKUM PEKAN 5
STRUKTUR DATA
Single Linked List

Disusun oleh:
Thaariq Salam
2511532022

Dosen Pengampu: Dr. Wahyudi S.T.M.T
Asisten Praktikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas rahmat-Nya sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data mengenai materi "*Single Linked List*". Laporan ini disusun untuk mendokumentasikan hasil pengerjaan serta pemahaman penulis terhadap mekanisme penyimpanan data dinamis menggunakan *pointer* dalam pemrograman Java.

Melalui praktikum ini, penulis telah mempelajari berbagai cara mengimplementasikan *Single Linked List*, mulai dari representasi simpul, operasi penambahan di berbagai posisi, hingga mekanisme penghapusan simpul. Penulis menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi S.T.M.T selaku Dosen Pengampu mata kuliah Struktur Data.
2. Kak Sherly Sukmadira selaku Asisten Praktikum yang telah memberikan arahan teknis selama pelaksanaan praktikum.

Saya menyadari sepenuhnya bahwa laporan praktikum ini masih jauh dari kata sempurna. Terdapat keterbatasan dalam analisis maupun teknis penulisan yang mungkin ditemukan. Oleh karena itu, saya sangat mengharapkan kritik yang membangun serta saran dari pembaca demi perbaikan di masa yang akan datang.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi saya pribadi maupun bagi pembaca yang ingin mendalami konsep pemrograman Java.

Padang, 7 Mei 2026

Thaariq Salam

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Class NodeSLL_2511532022.java.....	3
2.1.1 Kode Program.....	3
2.1.2 Langkah Kerja.....	3
2.1.3 Analisis dan <i>Output</i>	4
2.2 Class TambahSLL_2511532022.java.....	4
2.2.1 Kode Program.....	4
2.2.2 Langkah Kerja.....	7
2.2.3 Analisis dan <i>Output</i>	7
2.3 Class PencariSLL_2511532022.java.....	8
2.3.1 Kode program.....	8
2.3.2 Langkah Kerja.....	9
2.3.3 Analisis dan <i>Output</i>	10
2.4 Class HapusSLL_2511532022.java.....	10
2.4.1 Kode program.....	10
2.4.2 Langkah Kerja.....	13
2.4.3 Analisis dan <i>Output</i>	13
BAB III PENUTUP.....	15
3.1 Kesimpulan.....	15
3.2 Saran.....	15
TUGAS PEKAN 5.....	16
1. Deskripsi Tugas.....	16
2. Kode Program.....	16
2.1 Kode Program <i>ADT Class (Pasién_2511532022.java)</i>	16
2.2 Kode Program <i>Driver Class (RumahSakit_2511532022.java)</i>	17
3. Langkah Kerja.....	21
4. Output dan Analisis.....	22
4.1 Screenshoot Output.....	22
4.2 Analisis.....	24

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia informatika, pengelolaan data yang efisien merupakan kunci utama dalam pembangunan perangkat lunak yang optimal. Salah satu konsep dasar yang wajib dipahami adalah struktur data linier, di mana elemen-elemen data disusun secara berurutan. Di antara berbagai jenis struktur data, *Linked List* menjadi sangat penting karena karakteristik uniknya yang menggunakan alokasi memori secara dinamis, berbeda dengan *Array* yang bersifat statis dan memerlukan alokasi memori secara kontigu (berurutan).

Penerapan *Single Linked List* (SLL) sangat luas dalam komputasi modern, mulai dari manajemen memori pada sistem operasi, implementasi struktur data lain seperti *Stack* dan *Queue*, hingga pemrosesan simbol pada kompilator. Dalam lingkungan Java, struktur ini memungkinkan pengembang untuk menambah atau menghapus elemen tanpa harus melakukan pergeseran elemen (*shifting*) yang memakan waktu, sehingga sangat fleksibel dalam menangani jumlah data yang berubah-ubah secara drastis saat *runtime*.

Laporan ini disusun sebagai bentuk dokumentasi atas hasil pemahaman teknis terhadap konsep *Single Linked List* yang telah dipraktikkan. Fokus utama dari laporan ini adalah pemahaman mekanisme simpul (*node*) dan pengaturan alamat memori melalui *pointer next*, guna memahami bagaimana alur data dapat dikelola secara sistematis dan efisien.

1.2 Tujuan

1. Memahami prinsip kerja struktur data *Single Linked List* melalui representasi simpul dan keterhubungan antar memori.
2. Mampu mengimplementasikan operasi penambahan simpul pada posisi depan, belakang, dan tengah secara manual.

3. Mampu melakukan manipulasi data pada list, termasuk pencarian kunci (*key*) dan penelusuran (*traversal*) elemen.
4. Memahami teknik penghapusan simpul dengan mengelola pemutusan *link* agar tidak terjadi kebocoran memori (*memory leak*).

1.3 Manfaat

1. Meningkatkan kemampuan analisis dalam memilih struktur data yang tepat untuk kasus yang membutuhkan fleksibilitas ukuran data yang tinggi
2. Memahami logika referensi memori di Java melalui penggunaan variabel pointer yang menghubungkan satu objek dengan objek lainnya
3. Melatih ketelitian dalam mengelola kondisi khusus dalam pemrograman seperti list kosong atau posisi di luar jangkauan
4. Membiasakan standarisasi penulisan kode yang baik dengan menggunakan identitas pribadi pada setiap variabel yang digunakan.

BAB II

PEMBAHASAN

2.1 Class `NodeSLL_2511532022.java`

2.1.1 Kode Program

```
package pekan5_2511532022;  
public class NodeSLL_2511532022 {  
    int data_2022;  
    NodeSLL_2511532022 next_2022;  
  
    public NodeSLL_2511532022 (int data_2022) {  
        this.data_2022 = data_2022;  
        this.next_2022 = null;  
    }  
}
```

2.1.2 Langkah Kerja

- 1) Baris `int data_2022` merupakan deklarasi variabel instansi untuk menyimpan nilai numerik yang menjadi muatan data utama di dalam simpul
- 2) Baris `NodeSLL_2511532022 next_2022` mendefinisikan sebuah referensi objek ke tipe yang sama yang berfungsi sebagai pointer untuk menunjuk ke simpul selanjutnya di dalam memori
- 3) Baris `public NodeSLL_2511532022` adalah konstruktor yang digunakan untuk menciptakan objek simpul baru dengan satu parameter masukan data
- 4) Baris `this.data_2022 = data_2022` memberikan nilai dari parameter ke variabel instansi simpul tersebut
- 5) Baris `this.next_2022 = null` mengatur nilai awal pointer ke null yang menandakan bahwa saat diciptakan simpul ini belum terhubung ke simpul manapun

2.1.3 Analisis dan Output

Class ini mendefinisikan struktur fundamental dari sebuah *node*. Penggunaan tipe data objek untuk variabel *next_2022* memungkinkan terjadinya hubungan berantai di memori. Simpul ini merupakan dasar dari alokasi dinamis di mana memori hanya akan digunakan saat objek simpul ini diinstansiasi.

Output dari *class* ini biasanya tidak terlihat secara langsung di konsol karena kelas ini berfungsi sebagai *blueprint*. Namun, saat objek dibuat di memori, simpul akan memiliki struktur data yang menyimpan nilai tertentu dan sebuah alamat *null* yang siap disambungkan ke node lain.

2.2 Class TambahSLL_2511532022.java

2.2.1 Kode Program

```
package pekan5_2511532022;

public class TambahSLL_2511532022 {
    public static NodeSLL_2511532022
insertAtFront(NodeSLL_2511532022 head_2022, int value_2022) {
        NodeSLL_2511532022 new_node = new
NodeSLL_2511532022(value_2022);
        new_node.next_2022= head_2022; // Menghubungkan node
baru ke head lama
        return new_node;
    }

    public static NodeSLL_2511532022 insertAtEnd
(NodeSLL_2511532022 head_2022, int value_2022) {
        NodeSLL_2511532022 newNode = new
NodeSLL_2511532022(value_2022);
        if (head_2022 == null) { // Kondisi jika list kosong
            return newNode;
        }
        NodeSLL_2511532022 last = head_2022;
        while(last.next_2022 != null) { // Iterasi mencari
node terakhir
            last = last.next_2022;
```



```

    }
    last.next_2022 = newNode; // Menyambungkan node
    terakhir ke node baru
    return head_2022;
}

static NodeSLL_2511532022 GetNode (int data_2022) {
    return new NodeSLL_2511532022(data_2022);
}

static NodeSLL_2511532022 insertPos(NodeSLL_2511532022
headNode_2022, int position_2022, int value_2022) {
    NodeSLL_2511532022 head_2022 = headNode_2022;
    if(position_2022 <1 ) {
        System.out.print("invalid position");
    }
    if (position_2022 == 1) { // Penanganan jika posisi
    adalah head
        NodeSLL_2511532022 new_node_2022 = new
NodeSLL_2511532022 (value_2022);
        new_node_2022.next_2022 = head_2022;
        return new_node_2022;
    } else {
        while (position_2022 -- != 0) {
            if (position_2022 == 1) { // Penyisipan di
posisi target
                NodeSLL_2511532022 newNode =
GetNode(value_2022);
                newNode.next_2022 =
headNode_2022.next_2022;
                headNode_2022.next_2022 = newNode ;
                break;
            }
            headNode_2022 = headNode_2022.next_2022;
        }
        if (position_2022 != 1)
            System.out.print("posisi di luar
jangkauan");
    }
}

```

```

        return head_2022;
    }

    public static void printList (NodeSLL_2511532022 head_2022)
    {
        NodeSLL_2511532022 curr = head_2022;
        while (curr.next_2022 != null) {
            System.out.print (curr.data_2022 + "-->");
            curr = curr.next_2022;
        }
        if (curr.next_2022== null) {
            System.out.print(curr.data_2022);
        }
        System.out.println();
    }

    public static void main (String[]args) {
        NodeSLL_2511532022 head_2022 = new
NodeSLL_2511532022(2); // Inisialisasi awal
        head_2022.next_2022 = new NodeSLL_2511532022 (5);
        head_2022.next_2022.next_2022 = new NodeSLL_2511532022
(6);

        System.out.print("senarai berantai awal: ");
        printList(head_2022);

        System.out.print("tambah 1 simpul didepan: ");
        head_2022 = insertAtFront(head_2022, 1);
        printList (head_2022);

        System.out.print("tambah 1 simpul dibelakang: ");
        head_2022 = insertAtEnd(head_2022, 7);
        printList(head_2022);

        System.out.print("tambah 1 simpul ke data 4: ");
        head_2022 = insertPos(head_2022, 4, 4);
        printList(head_2022);
    }
}

```

2.2.2 Langkah Kerja

- 1) Metode *insertAtFront* bekerja dengan membuat node baru kemudian mengarahkan pointer next simpul tersebut ke arah head lama agar ia menjadi elemen paling awal
- 2) Metode *insertAtEnd* menggunakan perulangan while untuk menelusuri list hingga menemukan simpul yang memiliki nilai next null lalu menyambungkan simpul baru di sana
- 3) Metode *insertPos* melakukan penelusuran alamat memori sesuai angka posisi yang diinputkan kemudian menyisipkan node baru dengan mengatur ulang rantai pointer agar tidak terputus
- 4) Metode *printList* bertugas menampilkan data secara berurutan dengan menggunakan simbol panah sebagai visualisasi hubungan logis antar simpul di memori

2.2.3 Analisis dan Output

```
<terminated> TambahSLI_2511532022 [Java Application] /app/ect  
senarai berantai awal: 2-->5-->6  
tambah 1 simpul didepan: 1-->2-->5-->6  
tambah 1 simpul dibelakang: 1-->2-->5-->6-->7  
tambah 1 simpul ke data 4: 1-->2-->5-->4-->6-->7
```

Analisis dari output di atas menunjukkan transformasi data secara bertahap di mana list diawali dengan rangkaian 2-->5-->6 lalu penambahan angka 1 di depan mengubahnya menjadi 1-->2-->5-->6. Selanjutnya penambahan angka 7 di belakang menghasilkan 1-->2-->5-->6-->7 dan terakhir penyisipan angka 4 pada posisi 4 secara presisi menempatkan data di antara angka 5 dan 6 sehingga hasil akhirnya adalah 1-->2-->5-->4-->6-->7.

2.3 Class PencariSLL_2511532022.java

2.3.1 Kode program

```
package pekan5_2511532022;

public class PencariSLL_2511532022 {
    static boolean searchKey (NodeSLL_2511532022 head_2022, int
key_2022) {
        NodeSLL_2511532022 curr = head_2022; // Inisialisasi
kursor penelusuran
        while (curr != null) {
            if (curr.data_2022 == key_2022)
                return true; // Mengembalikan true jika
data ditemukan
            curr = curr.next_2022; // Geser ke simpul
berikutnya
        }
        return false; // Mengembalikan false jika tidak
ditemukan
    }

    public static void traversal(NodeSLL_2511532022 head_2022)
{
        NodeSLL_2511532022 curr = head_2022;
        while (curr != null) {
            System.out.print(" " + curr.data_2022);
            curr = curr.next_2022; // Navigasi antar alamat
memori
        }
        System.out.println();
    }

    public static void main (String[]args) {
        // Inisialisasi rangkaian Single Linked List
        NodeSLL_2511532022 head_2022 = new NodeSLL_2511532022
(14);
        head_2022.next_2022 = new NodeSLL_2511532022 (21);
        head_2022.next_2022.next_2022 = new NodeSLL_2511532022
(13);
```

```

        head_2022.next_2022.next_2022.next_2022      =      new
NodeSLL_2511532022  (30);
        head_2022.next_2022.next_2022.next_2022.next_2022      =
new NodeSLL_2511532022  (10);

        System.out.print(" penelusuran SLL : ");
        traversal (head_2022);

        int key_2022 = 30; // Data yang dicari
        System.out.print(" cari data " + key_2022 + " = ");

        if (searchKey(head_2022,key_2022)) {
            System.out.println("ketemu");
        } else {
            System.out.println("tidak ada");
        }
    }
}

```

2.3.2 Langkah Kerja

- 1) Baris *NodeSLL_2511532022 curr = head_2022* berfungsi untuk menginisialisasi variabel bantu agar proses penelusuran dimulai dari simpul paling depan tanpa mengubah alamat asli dari kepala list
- 2) Baris *while (curr != null)* digunakan sebagai batas perulangan agar program terus memeriksa isi simpul selama kursor belum mencapai alamat kosong di ujung rangkaian
- 3) Baris *if (curr.data_2022 == key_2022)* merupakan logika perbandingan untuk mencocokkan nilai data pada simpul aktif dengan nilai kunci yang sedang dicari oleh pengguna
- 4) Baris *curr = curr.next_2022* bertugas menggeser posisi kursor ke alamat simpul berikutnya melalui referensi pointer yang tersimpan pada atribut next

2.3.3 Analisis dan Output

```
<terminated> PencariSLL_2511532022 [Java Application]
penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu
```

Analisis dari hasil eksekusi program di atas menunjukkan bahwa metode *traversal* berhasil menampilkan seluruh urutan data yaitu 14 21 13 30 10. Ketika fungsi *searchKey* dipanggil untuk mencari angka 30 maka sistem melakukan penyisiran alamat memori satu per satu hingga mencapai simpul keempat. Karena nilai data ditemukan cocok maka program memberikan balasan berupa teks "ketemu" yang membuktikan bahwa mekanisme navigasi *pointer* pada *Single Linked List* telah berfungsi dengan benar.

2.4 Class HapusSLL_2511532022.java

2.4.1 Kode program

```
package pekan5_2511532022;
```

```
public class HapusSLL_2511532022 {
    public static NodeSLL_2511532022 deleteHead
(NodeSLL_2511532022 head_2022) {
        if (head_2022 == null)
            return null;
        head_2022 = head_2022.next_2022; // Menggeser head ke
simpul berikutnya
        return head_2022;
    }

    public static NodeSLL_2511532022 removeLastNode
(NodeSLL_2511532022 head_2022) {
        if (head_2022 == null) {
            return null;
        }
        if (head_2022.next_2022 == null) {
            return null;
        }
    }
}
```

```

        NodeSLL_2511532022 secondLast = head_2022; // Mencari
node sebelum terakhir
        while (secondLast.next_2022.next_2022 != null) {
            secondLast = secondLast.next_2022;
        }
        secondLast.next_2022 = null; // Memutus rantai
terakhir
        return head_2022;
    }

```

```

        public static NodeSLL_2511532022
deleteNode( NodeSLL_2511532022 head_2022, int position_2022) {
    NodeSLL_2511532022 temp = head_2022;
    NodeSLL_2511532022 prev = null;

    if (temp == null)
        return head_2022;

    if (position_2022 == 1) { // Kondisi jika yang dihapus
adalah head
        head_2022 = temp.next_2022;
        return head_2022;
    }

    for (int i=1; temp != null && i< position_2022; i++) {
        prev = temp; // Menyimpan simpul sebelum target
        temp = temp.next_2022; // Mencari simpul target
    }

    if (temp != null) {
        prev.next_2022 = temp.next_2022; // Menghubungkan
simpul prev ke sesudah target
    } else {
        System.out.println("Data tidak ada");
    }
    return head_2022;
}

```

```

public static void printList (NodeSLL_2511532022 head_2022)
{
    NodeSLL_2511532022 curr = head_2022;
    while (curr.next_2022 != null) {
        System.out.print(curr.data_2022 + "--->");
        curr = curr.next_2022;
    }
    if (curr.next_2022 == null) {
        System.out.print(curr.data_2022);
    }
    System.out.println();
}

public static void main (String []args) {
    // Inisialisasi awal list 1 sampai 6
    NodeSLL_2511532022 head_2022 = new NodeSLL_2511532022
(1);

    head_2022.next_2022 = new NodeSLL_2511532022 (2);
    head_2022.next_2022.next_2022 = new NodeSLL_2511532022
(3);

    head_2022.next_2022.next_2022.next_2022 = new
NodeSLL_2511532022 (4);
    head_2022.next_2022.next_2022.next_2022.next_2022 =
new NodeSLL_2511532022 (5);

    head_2022.next_2022.next_2022.next_2022.next_2022.next_2022
= new NodeSLL_2511532022 (6);

    System.out.print("list awal: ");
    printList(head_2022);

    head_2022 = deleteHead(head_2022); // Operasi hapus
depan

    System.out.print("List setelah head dihapus: ");
    printList(head_2022);

    head_2022= removeLastNode(head_2022); // Operasi hapus
belakang

    System.out.print("List setelah simpul terakhir: ");

```



```

        printList(head_2022);

        int position = 2;
        head_2022 = deleteNode(head_2022, position); //
Operasi hapus posisi tertentu

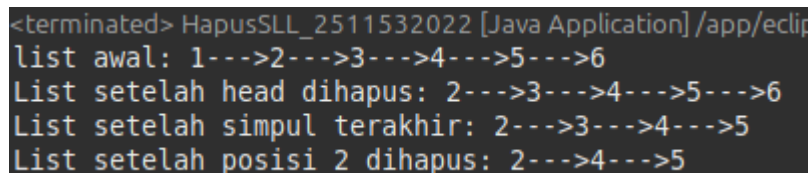
        System.out.print("List setelah posisi 2 dihapus: ");
        printList(head_2022);
    }
}

```

2.4.2 Langkah Kerja

- 1) Metode *deleteHead* dilakukan dengan mengalihkan referensi *head_2022* langsung ke simpul di belakangnya sehingga simpul pertama tidak lagi memiliki akses di dalam *list*.
- 2) Metode *removeLastNode* mencari simpul kedua terakhir menggunakan bantuan variabel *secondLast* kemudian memutus hubungan *pointer*-nya dengan mengubah nilai *next* menjadi *null*.
- 3) Metode *deleteNode* menggunakan perulangan untuk menemukan simpul target berdasarkan angka posisi lalu menghubungkan simpul sebelumnya (*prev*) langsung ke simpul sesudah target agar data target terisolasi.
- 4) Program menyertakan validasi kondisi *list* kosong serta pengecekan ketersediaan data untuk memastikan manipulasi alamat *pointer* tidak menyebabkan *error* saat aplikasi dijalankan.

2.4.3 Analisis dan Output



```

<terminated> HapusSLL_2511532022 [Java Application] /app/eclip
list awal: 1--->2--->3--->4--->5--->6
List setelah head dihapus: 2--->3--->4--->5--->6
List setelah simpul terakhir: 2--->3--->4--->5
List setelah posisi 2 dihapus: 2--->4--->5

```

Analisis dari hasil output di atas menunjukkan proses modifikasi list yang dimulai dari urutan 1--->2--->3--->4--->5--->6. Setelah *deleteHead* dipanggil

simpul angka 1 dihapus menghasilkan list 2--->3--->4--->5--->6. Penghapusan simpul terakhir melalui `removeLastNode` membuang angka 6 sehingga menyisakan 2--->3--->4--->5. Terakhir penghapusan pada posisi 2 membuang angka 3 sehingga hasil akhir yang ditampilkan adalah 2--->4--->5.

Hal ini mengonfirmasi bahwa seluruh logika manipulasi alamat memori untuk penghapusan data telah berhasil dijalankan dengan tepat.

BAB III

PENUTUP

3.1 Kesimpulan

Setelah saya melaksanakan praktikum mengenai *Single Linked List* dan melakukan pengujian langsung terhadap kode program yang telah disusun, saya dapat menyimpulkan bahwa:

- Melalui praktikum ini saya memahami bahwa *Single Linked List* jauh lebih fleksibel dalam penggunaan memori dibandingkan array karena saya dapat menambah data secara dinamis tanpa perlu menentukan kapasitas maksimal sejak awal
- Pada saat melakukan operasi penambahan data saya menemukan bahwa memanipulasi pointer next secara urut sangat penting agar simpul baru dapat masuk ke dalam rangkaian tanpa menghilangkan referensi ke simpul yang sudah ada sebelumnya
- Dalam proses penghapusan simpul saya belajar bahwa saya harus secara manual memindahkan rujukan pointer agar objek yang ingin dihapus benar-benar terisolasi dan dapat dibersihkan dari memori sistem

3.2 Saran

Berdasarkan kendala dan pengalaman yang saya temukan selama proses praktikum ini saya ingin memberikan beberapa saran untuk pengembangan selanjutnya:

- Saya menyadari perlunya bagi saya untuk mencoba implementasi *Double Linked List* agar saya bisa membandingkan perbedaan efisiensi saat harus melakukan penelusuran data dari arah belakang.
- Menurut saya penggunaan diagram visual sangat membantu saya sebelum mulai menulis kode sehingga saya memiliki gambaran yang jelas mengenai arah perpindahan alamat pointer dalam memori.

TUGAS PEKAN 5

Topik : Implementasi *Single Linked List*

Tema : Simulasi Antrian Rumah Sakit

1. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep *Single Linked List* yang bekerja secara dinamis menggunakan *pointer* (referensi antar *node*). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah *node* yang memiliki *pointer* ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (*FIFO — First In, First Out*).

2. Kode Program

2.1 Kode Program ADT Class (*Pasien_2511532022.java*)

Kelas ini berfungsi sebagai struktur data atau *Node* yang menyimpan informasi pasien dan referensi ke pasien berikutnya.

```
package tugas_2511532022;
public class Pasien_2511532022 {
    // Atribut wajib dengan akhiran 4 digit NIM
    private String namaPasien_2022;
    private String keluhan_2022;
    private int nomorAntrian_2022;
    Pasien_2511532022 next_2022; // Pointer ke node berikutnya

    // Constructor untuk menginisialisasi atribut
    public Pasien_2511532022(String nama_2022, String sakit_2022,
int antrian_2022) {
        this.namaPasien_2022 = nama_2022;
        this.keluhan_2022 = sakit_2022;
        this.nomorAntrian_2022 = antrian_2022;
        this.next_2022 = null;
    }
}
```

```

// Selektor (Getter)
public String getNama_2022() { return namaPasien_2022; }
public String getKeluhan_2022() { return keluhan_2022; }
public int getNomor_2022() { return nomorAntrian_2022; }
public Pasien_2511532022 getNext_2022() { return next_2022; }

// Mutator (Setter)
public void setNama_2022(String n) { this.namaPasien_2022 = n;
}
public void setKeluhan_2022(String k) { this.keluhan_2022 = k;
}
public void setNext_2022(Pasien_2511532022 nxt)
{ this.next_2022 = nxt; }
}

```

2.2 Kode Program *Driver Class* (*RumahSakit_2511532022.java*)

Kelas ini menangani seluruh logika operasional antrian seperti menambah, memanggil, dan menampilkan data.

```

package pekan5_2511532022;
import java.util.Scanner;

public class RumahSakit_2511532022 {
    private Pasien_2511532022 head_2022 = null;
    private int counterAntrian_2022 = 0; // Auto-increment nomor
    antrian

    // 1. Daftarkan Pasien (Insert at Tail) (ekor)
    public void daftarkan_2022(String nama, String keluhan) {
        counterAntrian_2022++;
        Pasien_2511532022 baru = new Pasien_2511532022(nama,
        keluhan, counterAntrian_2022);

        if (head_2022 == null) {
            head_2022 = baru; // Jika list kosong, simpul baru
            jadi head
        } else {

```

```

        Pasien_2511532022 temp = head_2022;
        while (temp.next_2022 != null) { // Iterasi sampai
ekor (tail)
            temp = temp.next_2022;
        }
        temp.next_2022 = baru; // Menyambungkan di akhir
    }
    System.out.println("Pasien berhasil didaftarkan! Nomor
Antrian: " + counterAntrian_2022);
}

// 2. Panggil Pasien (Delete Head)
public void panggil_2022() {
    if (head_2022 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    System.out.println("Memanggil Pasien: " +
head_2022.getNama_2022());
    System.out.println("Keluhan: " +
head_2022.getKeluhan_2022());
    head_2022 = head_2022.next_2022; // Menggeser head ke
simpul berikutnya
}

// 3. Tampilkan Antrian (Display)
public void tampilkan_2022() {
    if (head_2022 == null) {
        System.out.println("Antrian kosong.");
        return;
    }
    Pasien_2511532022 curr = head_2022;
    System.out.println("\n--- Daftar Antrian Saat Ini ---");
    while (curr != null) {
        System.out.println "[" + curr.getNomor_2022() + "] " +
curr.getNama_2022() + " (" + curr.getKeluhan_2022() + ")";
        curr = curr.next_2022;
    }
}
}

```

```

// 4. Cari Pasien (Search - Case Insensitive)
public void cari_2022(String cariNama) {
    Pasien_2511532022 temp = head_2022;
    boolean ditemukan = false;
    while (temp != null) {
        if (temp.getNama_2022().equalsIgnoreCase(cariNama)) {
            System.out.println("Data Ditemukan: " +
temp.getNama_2022() + " berada di Antrian No: " +
temp.getNomor_2022());
            ditemukan = true;
        }
        temp = temp.next_2022;
    }
    if (!ditemukan) System.out.println("Pasien dengan nama '"
+ cariNama + "' tidak ditemukan.");
}

// 5. Cek Status Antrian
public void cekStatus_2022() {
    if (head_2022 == null) {
        System.out.println("Antrian saat ini masih kosong.");
        return;
    }
    int total = 0;
    Pasien_2511532022 temp = head_2022;
    while (temp != null) {
        total++;
        temp = temp.next_2022;
    }
    System.out.println("Total Pasien dalam antrian: " +
total);
    System.out.println("Pasien terdepan: " +
head_2022.getNama_2022());
}

public static void main(String[] args) {
    RumahSakit_2511532022 rs = new RumahSakit_2511532022();
    Scanner sc = new Scanner(System.in);
    int pilih;
    do {

```

```

        System.out.println("\n=== Antrian Rumah Sakit NIM:
2511532022 ===");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilih = sc.nextInt();
        sc.nextLine(); // consume newline

        switch (pilih) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String n = sc.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String k = sc.nextLine();
                rs.daftarkan_2022(n, k);
                break;
            case 2:
                rs.panggil_2022();
                break;
            case 3:
                rs.tampilkan_2022();
                break;
            case 4:
                System.out.print("Cari Nama: ");
                String cn = sc.nextLine();
                rs.cari_2022(cn);
                break;
            case 5:
                rs.cekStatus_2022();
                break;
        }
    } while (pilih != 6);
}
}

```


3. Langkah Kerja

Saya merancang kelas *Pasien_2511532022* sebagai struktur dasar *node* yang menyimpan data pasien beserta *pointer next_2022* untuk menghubungkan antar elemen antrian

- Saya membuat variabel *head_2022* pada kelas driver untuk menandai pasien mana yang berada di urutan paling depan dalam sistem rumah sakit
- Saya mengimplementasikan logika *Insert at Tail* pada metode pendaftaran agar setiap pasien baru yang masuk secara otomatis diletakkan pada posisi paling akhir dalam rangkaian memori
- Saya menerapkan mekanisme *Delete Head* untuk proses pemanggilan pasien guna memastikan sistem berjalan sesuai prinsip FIFO (*First In First Out*) di mana pasien yang mendaftar lebih awal akan dilayani terlebih dahulu
- Saya menggunakan perulangan *while* untuk melakukan penelusuran alamat memori dari simpul pertama hingga terakhir saat menjalankan fungsi tampilkan antrian dan pencarian pasien.

4. Output dan Analisis

4.1 Screenshoot Output

Daftarkan Pasien (*insert*)

```
<terminated> RumahSakit_2511532022 [Java Application] /app/
=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Thaariq
Masukkan Keluhan: Maag
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Kirito
Masukkan Keluhan: Diare
Pasien berhasil didaftarkan! Nomor Antrian: 2
```

Display Antrian dan Cek Pasien (*case insensitive*)

```
=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

--- Daftar Antrian Saat Ini ---
[1] Thaariq (Maag)
[2] Kirito (Diare)

=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Cari Nama: THAARIQ
Data Ditemukan: Thaariq berada di Antrian No: 1
```

Cek Pasien (*case insensitive*) dan Panggil Pasien (*delete head*)

```
=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Cari Nama: thaariq
Data Ditemukan: Thaariq berada di Antrian No: 1

=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien: Thaariq
Keluhan: Maag
```

Display Antrian setelah Delete Head

```
=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

--- Daftar Antrian Saat Ini ---
[2] Kirito (Diare)

=== Antrian Rumah Sakit NIM: 2511532022 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Total Pasien dalam antrian: 1
Pasien terdepan: Kirito
```

Menu Keluar

```
=== Antrian Rumah Sakit NIM: 2511532022 ===  
1. Daftarkan Pasien (Insert)  
2. Panggil Pasien (Delete Head)  
3. Tampilkan Antrian (Display)  
4. Cari Pasien (Search)  
5. Cek Status Antrian  
6. Keluar  
Pilihan: 6
```

4.2 Analisis

Berdasarkan hasil pengujian pada program simulasi ini, saya mengamati bahwa sistem mampu mengelola data antrian secara dinamis tanpa adanya batas kapasitas seperti pada array. Analisis saya menunjukkan bahwa penggunaan *Single Linked List* mempermudah proses manipulasi data pasien karena penambahan di posisi belakang dan penghapusan di posisi depan hanya melibatkan pemindahan referensi pointer. Hal ini membuktikan bahwa struktur data ini sangat efisien untuk diterapkan pada sistem antrian nyata yang memerlukan penambahan dan pengurangan data secara berkelanjutan.